THREAT HUNTING FRAMEWORKS · 02

Advanced Analytic Techniques for Threat Hunting

Eight ways to turn millions of events into a short list worth an analyst's time. Each technique answers one question, needs specific data, and has its own false-positive traps. Every section has a diagram, the maths in plain words, and a query you can adapt.

4.1 **Stack counting** — *What is rare here?*

4.2 **Time-series anomalies** — *Is this entity acting unlike itself?*

4.3 **Beaconing** — *Is something calling home on a timer?*

4.4 **Entropy & lexical** — *Was this name made by a machine?*

4.5 **Lineage & graphs** — *Who spawned this, and where did they go?*

4.6 **Peer groups** — *Is this unusual for its peers?*

4.7 **Sequences** — *Did these events happen in attack order?*

4.8 **Model-assisted** — *Which of millions should I look at?*

> **How to use this guide.** Most good hunts combine two or three of these. A typical chain: stack counting finds a rare binary, lineage shows what started it, the auth graph shows where the account went next, and a sequence query turns the whole pattern into a detection.

Counts, hosts, accounts, IP addresses and scores in the figures are illustrative. Entropy and Levenshtein values are calculated on the strings shown. Check field names in your own tenant before running the queries.

4.1 Stack counting and long-tail analysis

Group events by one attribute, count them, and sort so the **rarest values come first**. Attackers are few and legitimate software is everywhere, so malicious activity tends to sit in the long tail. It is the simplest technique and usually the first one to run on a new data source.

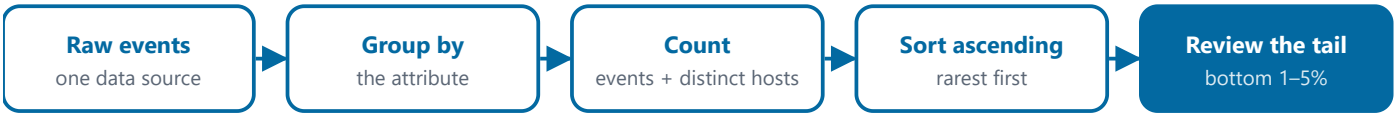


Figure 1. The stack counting workflow.

Parent → child pairs
winword → powershell, w3wp → cmd

User-writable paths
binaries in AppData, Temp, ProgramData, Downloads

Persistence across the fleet
services, scheduled tasks, Run keys, WMI subs

Unsigned DLLs in signed processes
DLL sideloading and hijacking

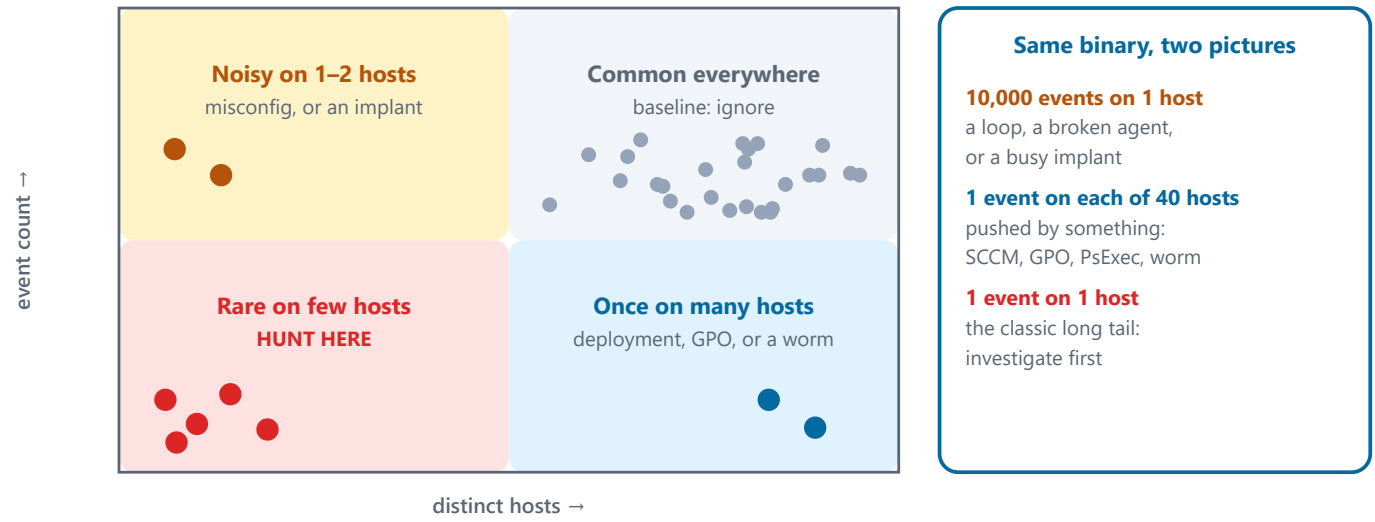


Figure 2. Always count distinct hosts as well as events: the two numbers together tell you what kind of rare you are looking at.

`CORTEX XDR · XQL · binaries running from user-writable paths, rarest first`

```
dataset = xdr_data
| filter event_type = ENUM.PROCESS and event_sub_type = ENUM.PROCESS_START
| filter action_process_image_path ~= "(?i)\\(Users\\[^\\]+\\
(AppData|Downloads|Desktop)|ProgramData|Windows\\Temp)\\"
| comp count() as runs, count_distinct(agent_hostname) as hosts by action_process_image_name,
action_process_signature_status
| sort asc hosts, asc runs
```

Hunter's tip. Normalise before you stack: strip GUIDs, PIDs, user names, version numbers and random temp folder names. Otherwise every row is unique and the long tail is the whole dataset.

4.2 Frequency and time-series anomaly detection

Compare each entity (user, host, service principal) with **its own history**. Build a baseline over 14–30 days, then flag days or hours that sit far outside it. A second form is **first-seen**: anything an entity has never done before in the lookback window.

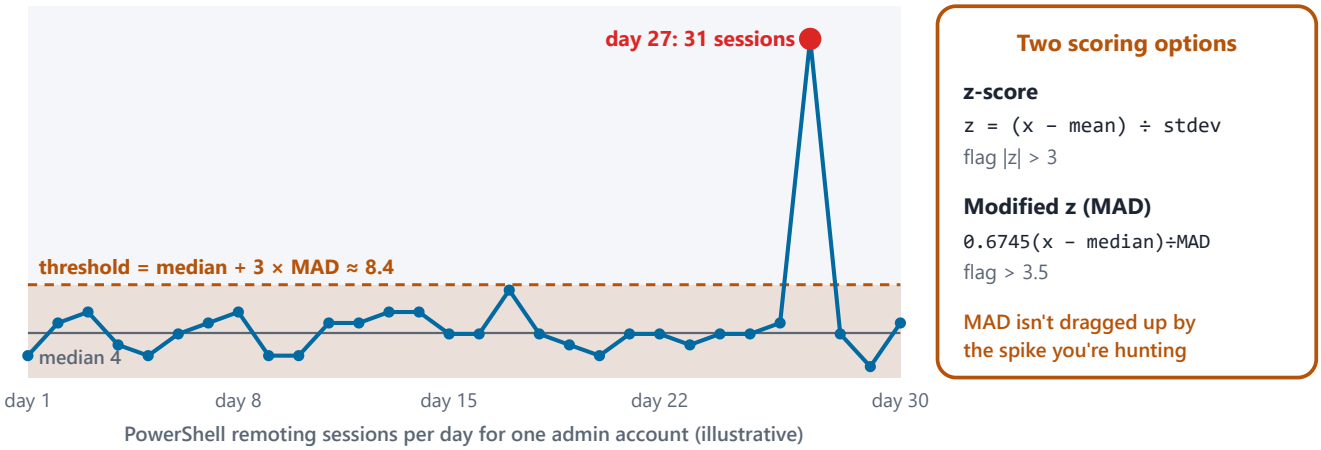


Figure 3. A daily series with a robust threshold. MAD-based scoring is safer than mean and standard deviation, because one big spike inflates the standard deviation and hides itself.

MICROSOFT DEFENDER / SENTINEL · KQL · native anomaly decomposition

```
DeviceLogonEvents
| where Timestamp > ago(30d) and LogonType == "Network"
| make-series logons = count() default = 0 on Timestamp step 1d by AccountName
| extend (anomaly, score, baseline) = series_decompose_anomalies(logons, 3.0)
| mv-expand Timestamp to typeof(datetime), logons to typeof(long), anomaly to typeof(int)
| where anomaly == 1
```

FIRST-SEEN LOGIC

Lookback: days −30 … −1 → build the set of what has been seen

Today
anything not in the set

- `user → wsmprovhost.exe on SRV07` — first PowerShell remoting by this user
- `app-sp-billing → sign-in from 203.0.113.50` — service principal, new source IP
- `WS-114 → schtasks /create` — first scheduled task from a workstation

Figure 4. First-seen hunting: no statistics needed, just a lookback set and a comparison.

KQL · first PowerShell remoting session on a host in 30 days

```
let seen = DeviceProcessEvents
    | where Timestamp between (ago(30d) .. ago(1d)) and FileName =~ "wsmprovhost.exe"
    | distinct DeviceName;
DeviceProcessEvents
| where Timestamp > ago(1d) and FileName =~ "wsmprovhost.exe"
| where DeviceName !in (seen)        // first PowerShell remoting session on this host
```

> **Hunter's tip.** Respect weekly cycles. A Monday spike is normal for many accounts, so compare Mondays with Mondays, or let series_decompose_anomalies remove the seasonal part for you.

4.3 Beaconing detection

C2 implants call home on a timer, with **jitter** added to look random. Over enough connections the timer still shows. The key measure is the **coefficient of variation (CV)** of the gaps between connections: standard deviation ÷ mean. Below about 0.2 points to automation.

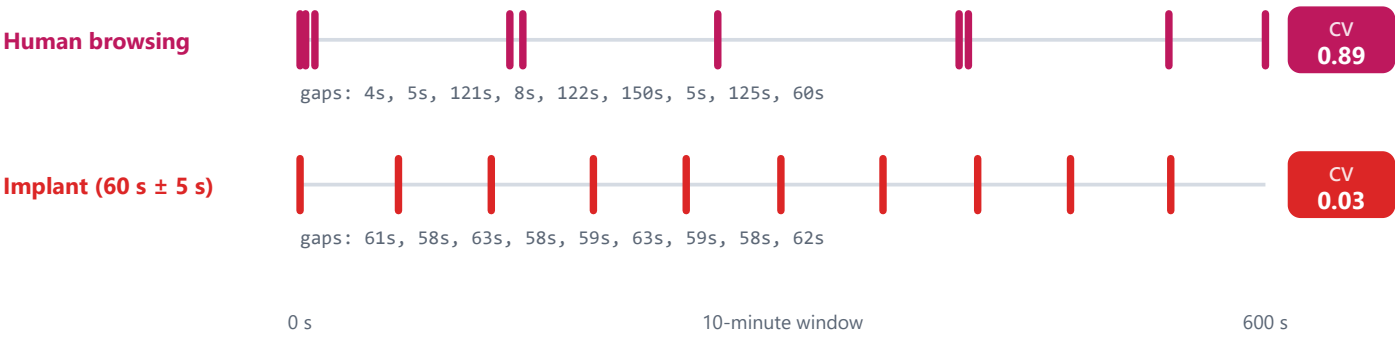


Figure 5. Ten connections each. Human traffic comes in bursts; the implant's gaps stay close to 60 seconds even with jitter.

Feature	What to measure	Why it matters
Connection count	Connections per src → dst:port in the window	Too few points and the CV is meaningless; require ≥ 30–50
Interval regularity	CV of time gaps; low (< 0.2) = automated	The core beacon signal, survives moderate jitter
Payload consistency	CV of bytes sent per connection	Check-ins with no tasking are the same size every time
Destination rarity	Number of internal hosts talking to the destination	Updaters talk to thousands of hosts; implants to one or two
Reputation and TLS	Domain age, reputation, JA3/JA4 fingerprint match	Raises confidence; known C2 frameworks have known fingerprints

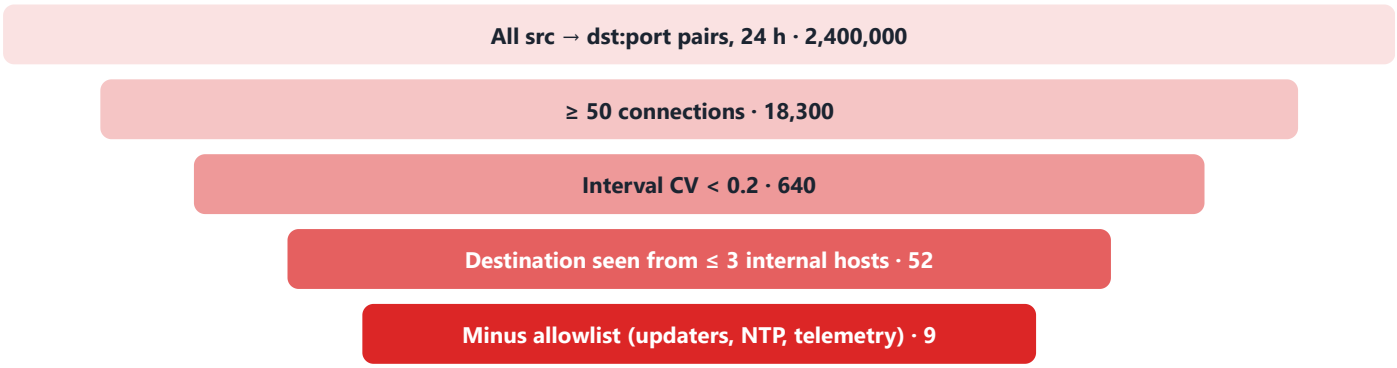


Figure 6. Apply the features as a funnel, cheapest filter first, and put the allowlist at the end.

PYTHON · pandas · score every pair (Zeek conn.log or firewall logs)

```
df = df.sort_values("ts")
key = ["src", "dst", "dport"]
df["gap"] = df.groupby(key)["ts"].diff().dt.total_seconds()
g = df.groupby(key).agg(n=("gap", "size"), gap_mean=("gap", "mean"), gap_sd=("gap", "std"),
                        size_mean=("bytes_out", "mean"), size_sd=("bytes_out", "std"))
g["cv"] = g.gap_sd / g.gap_mean                # interval regularity
g["size_cv"] = g.size_sd / g.size_mean         # payload consistency
rarity = df.groupby("dst")["src"].nunique().rename("src_hosts")
cand = (g.join(rarity, on="dst")
         .query("n >= 50 and cv < 0.2 and src_hosts <= 3")
         .drop(index=allowlist, errors="ignore")
         .sort_values("cv"))
```

Hunter's tip. Long sleep times (hours) and heavy jitter (50%+) defeat short windows. Run the same scoring over 7 days with a lower count threshold to catch slow beacons.

4.4 Entropy and lexical analysis

Machine-generated strings look different from human ones. **Shannon entropy** measures how random a string is: repeated letters lower it, many different characters raise it. Use it on domain labels (DGA), command-line arguments (encoded payloads), and service, task or file names.

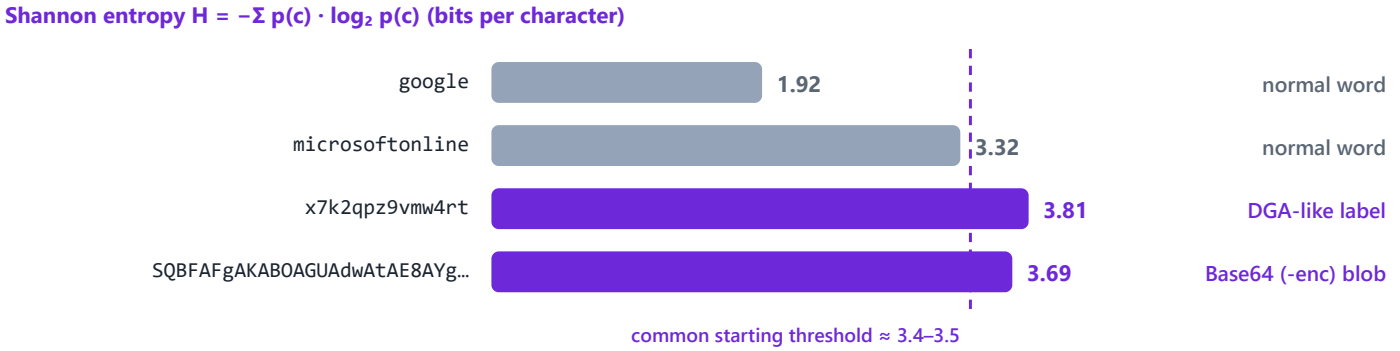


Figure 7. Entropy values calculated on real strings. Longer strings naturally score higher, so compare like with like.

PYTHON · entropy of the left-most domain label

```python
import math
from collections import Counter

def entropy(s: str) -> float:
    n = len(s)
    return -sum(c / n * math.log2(c / n) for c in Counter(s).values())

label = fqdn.split(".")[0]
suspicious = len(label) >= 10 and entropy(label) > 3.5
```



Figure 8. Levenshtein distance counts single-character edits. Distances of 1–2 from your brand domain are strong typo-squat candidates.

Lexical check	Human-made	Machine-made	How to score
Consonant / vowel ratio	WindowsUpdateSvc (11 ÷ 5 ≈ 2.2)	xvbrtqzk (no vowels)	Ratio > 3, or long runs of consonants
Character n-grams	“ing”, “ser”, “upd” are common	“qzx”, “vbk” almost never appear	Average bigram frequency against an English or top-sites corpus
Digit ratio	contoso365	a8f3k29d0q	Share of digits mixed through the label

> **Hunter's tip.** Entropy alone is noisy: CDNs, cloud storage and telemetry use random-looking subdomains. Combine entropy with rarity (few hosts query it) and domain age.

4.5 Process lineage and graph hunting

Single process events lose context. Rebuild the **full process tree** and hunt on unusual ancestry. Do the same with authentication: draw users and hosts as a graph, and lateral movement appears as a path.

Rebuilt process tree from a web server (Sysmon EID 1 or EDR)

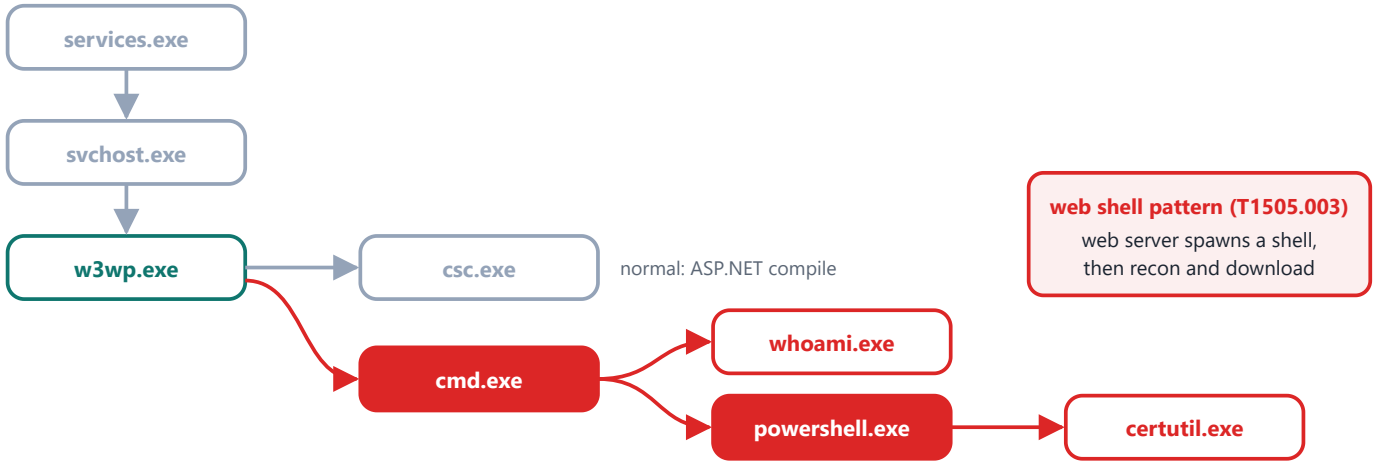


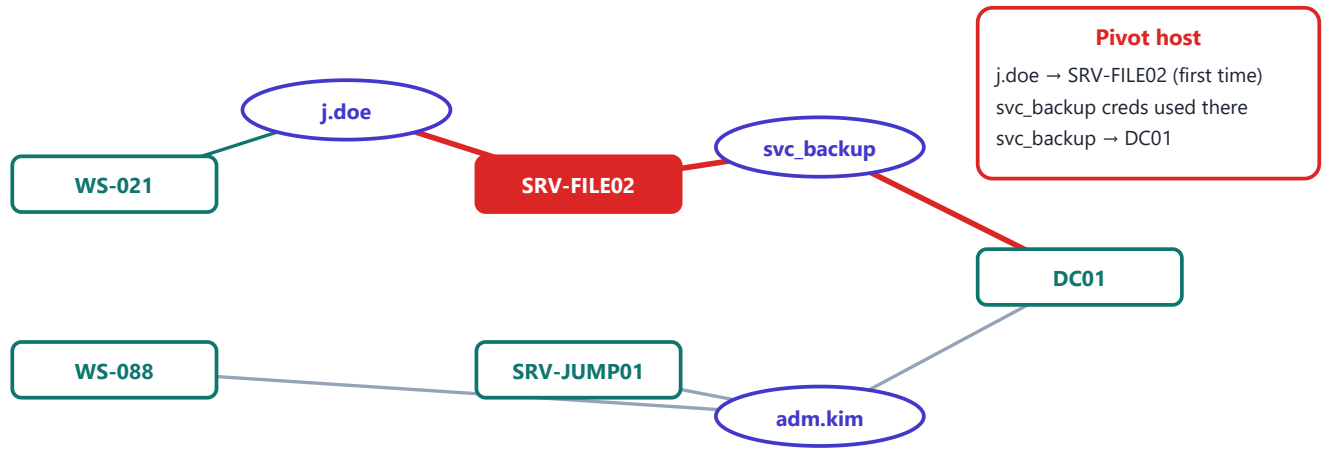
Figure 9. The same parent (w3wp.exe) has a normal child and a malicious branch. Lineage makes the difference obvious.

Suspicious lineage	Likely technique	Why it is unusual
`w3wp.exe → cmd.exe`	Web shell (T1505.003)	IIS workers serve pages; they don't need a shell
`sqlservr.exe → powershell.exe`	xp_cmdshell abuse (T1059)	Database engine running scripts on the host
`winword.exe → mshta.exe`	Malicious document (T1218.005)	Office launching an HTA runner
`services.exe → cmd.exe /c`	PsExec-style service (T1569.002)	A service whose binary is a one-shot shell command

CORTEX XDR · XQL · risky parent → child pairs

```
dataset = xdr_data
| filter event_type = ENUM.PROCESS and event_sub_type = ENUM.PROCESS_START
| filter actor_process_image_name in ("w3wp.exe", "sqlservr.exe", "winword.exe", "excel.exe")
    and action_process_image_name in ("cmd.exe", "powershell.exe", "mshta.exe", "rundll32.exe", "certutil.exe")
| comp count() as n, count_distinct(agent_hostname) as hosts by actor_process_image_name,
action_process_image_name
```

Authentication graph: who logged on where (4624 type 3/10, 4648)



□ host ○ account thick red line = the lateral movement path

Figure 10. BloodHound-style thinking for defence: the graph shows the pivot host where one account's access turned into another's.

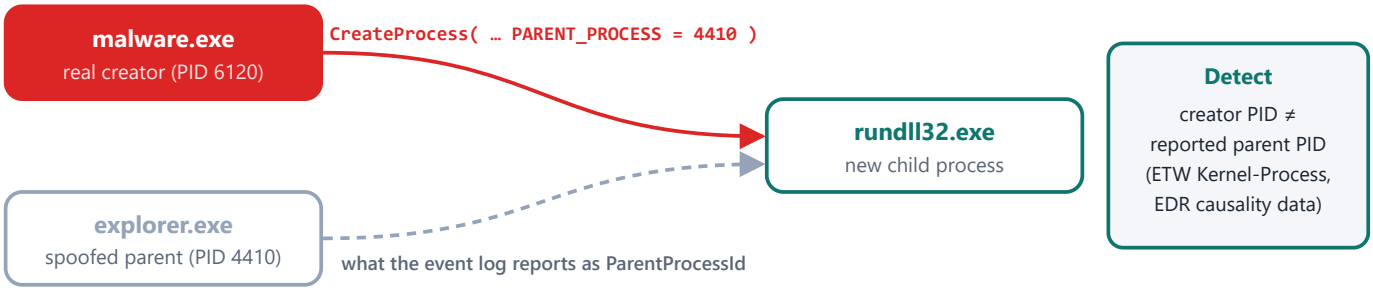


Figure 11. PPID spoofing. Tools such as Cobalt Strike can fake the parent; compare the real creator with the reported parent.

4.6 Peer-group analysis

Compare an entity with **its peers**, not the whole estate. IT admins touch hundreds of systems, so against everyone a busy finance user looks normal. Against other finance users the same behaviour stands out at once.

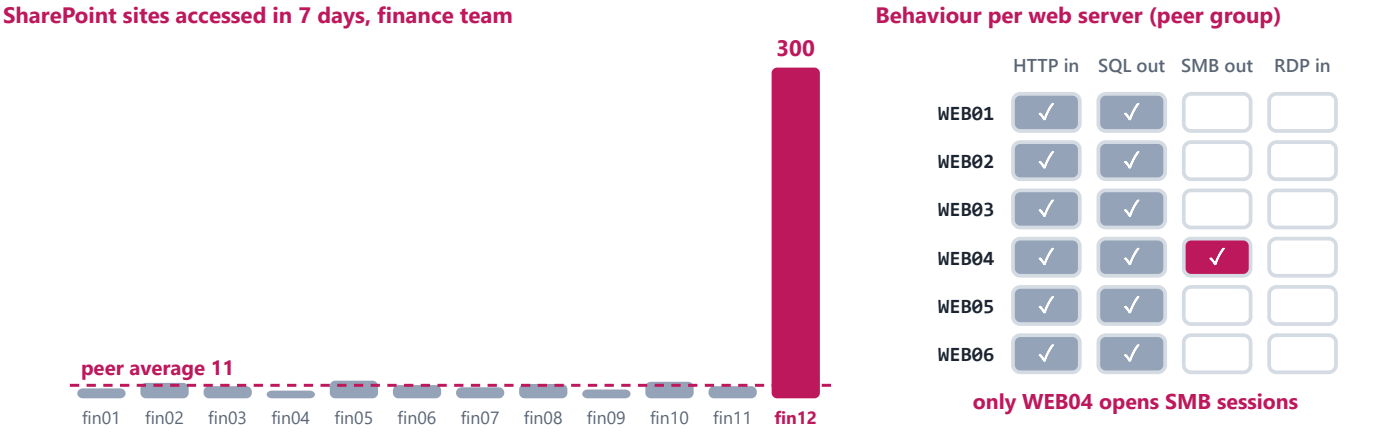


Figure 12. Left: one finance user accesses 25× more sites than the peer average. Right: one web server does something no other web server does.

> **Hunter's tip.** Build peer groups from HR or AD data (department, job title, OU) for users, and from role tags or naming conventions for servers. Bad peer groups produce bad results: check each group is genuinely similar before scoring against it.

4.7 Sequence and temporal correlation

Single events are noisy; **ordered sequences on the same entity within a short window** are strong signals. Each step alone may be benign. Together, in attack order, they rarely are.

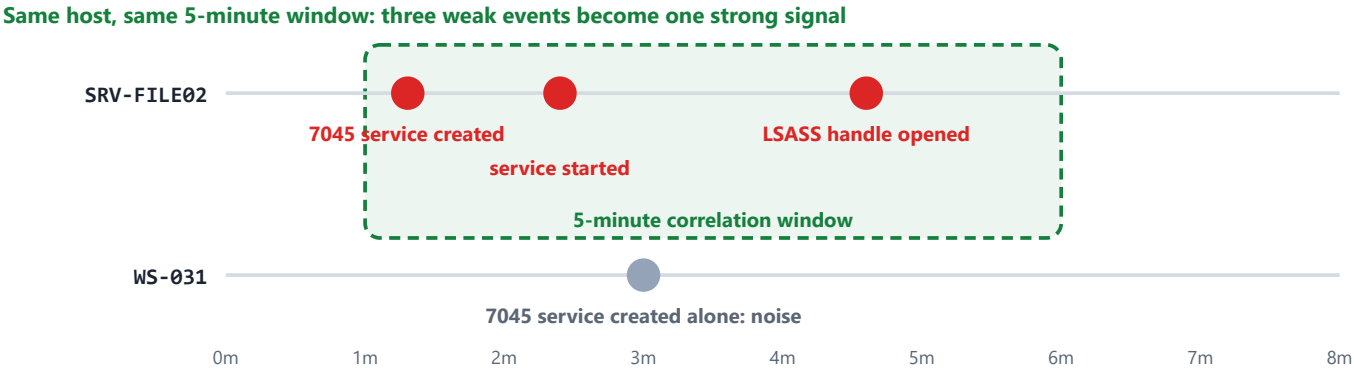


Figure 13. A service install happens on many hosts. A service install, its start and an LSASS handle within 5 minutes on one host does not.

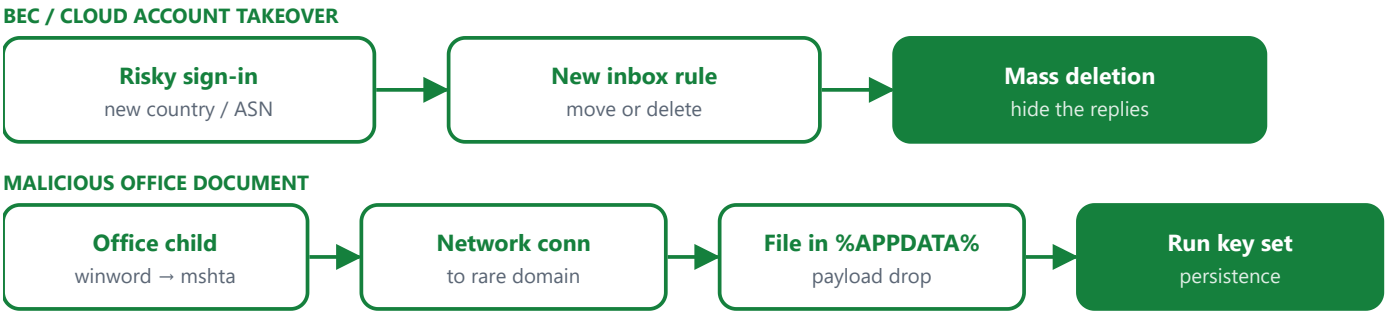


Figure 14. Two more sequences worth hunting: cloud account takeover and a malicious Office document.

`KQL · service installed, then LSASS opened within 5 minutes on the same device`

```
let svc = DeviceEvents
    | where ActionType == "ServiceInstalled"
    | project DeviceId, DeviceName, SvcTime = Timestamp, Service =
tostring(parse_json(AdditionalFields).ServiceName);
let lsass = DeviceEvents
    | where ActionType == "OpenProcessApiCall" and FileName =~ "lsass.exe"
    | project DeviceId, LsassTime = Timestamp, Opener = InitiatingProcessFileName;
svc
| join kind=inner lsass on DeviceId
| where LsassTime between (SvcTime .. SvcTime + 5m)
```

Platform	How to express a sequence
KQL	`join` on the entity plus a time-window filter, or `scan` for ordered state machines
Splunk SPL	`transaction` with `maxspan`, or `streamstats` by host for lighter-weight ordering
CrowdStrike CQL	`correlate()` for multi-event patterns, or `join()`
Cortex XQL	`join` between datasets on host and time, or causality chains in the XDR UI

4.8 Model-assisted hunting (M-ATH)

When the data is too large or the signal too subtle for stacking, let a model **generate candidates**. The analyst still gives every verdict.

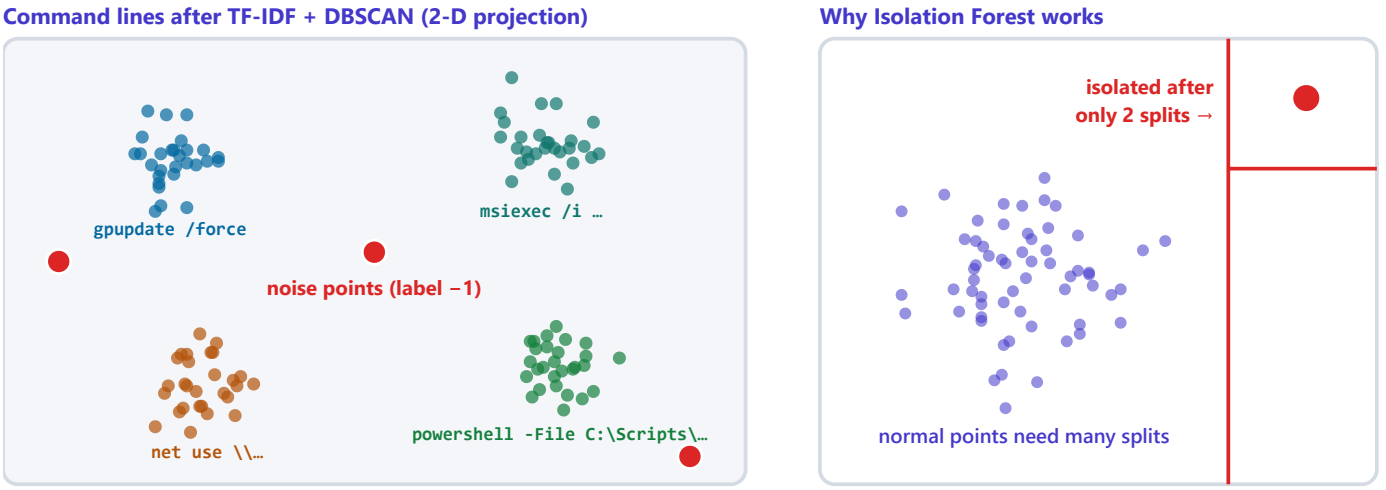


Figure 15. Left: DBSCAN groups similar command lines and leaves outliers unclustered. Right: Isolation Forest scores a point by how few random splits isolate it.

Method	Good for	Watch out for
k-means / DBSCAN	Grouping tokenised command lines; finding outlier clusters	k-means needs k chosen up front; DBSCAN is sensitive to eps
Isolation Forest	Multivariate anomalies: logon hour, source, volume together	Rare but benign admin work scores high; keep an allowlist
UEBA scores	Sentinel UEBA, Splunk UBA as a candidate generator	Black-box scores; always pull the raw events behind them
LLM-assisted triage	Summarising command lines, decoding obfuscation, drafting hypotheses	Can be confidently wrong; verify every claim against the data

PYTHON · scikit-learn · cluster command lines and pull the outliers

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import DBSCAN

X = TfidfVectorizer(token_pattern=r"[^\s\\/\"']+").fit_transform(cmdlines_normalised)
labels = DBSCAN(eps=0.4, min_samples=5, metric="cosine").fit_predict(X)
outliers = [c for c, l in zip(cmdlines_normalised, labels) if l == -1]   # review these
```



Figure 16. Every model output is a candidate. Only an analyst, looking at raw events, gives a verdict.

Cheat sheet: which technique, when?

Technique	Question it answers	Typical data	Main false-positive trap
Stack counting	What is rare across the fleet?	Process, autoruns, DLL loads	Unnormalised fields make everything rare
Time series	Is this entity unlike its own past?	Logons, sign-ins, process counts	Weekly cycles, holidays, new projects
First-seen	Has this ever happened before?	Any entity + action pair	New hires and new software
Beaconing	Is something calling home on a timer?	Zeek conn, proxy, firewall	Updaters, telemetry, NTP, health checks
Entropy and lexical	Was this name machine-generated?	DNS, command lines, task names	CDNs and cloud services with random names
Lineage and graphs	Who started this, and where did they go?	Process trees, 4624/4648	Admin tooling and deployment agents
Peer groups	Is this normal for its peers?	Any behaviour + HR/CMDB data	Badly defined peer groups
Sequences	Did events happen in attack order?	Several sources joined by entity	Windows too wide; clock skew between sources
M-ATH	Which few of millions deserve a look?	Large multivariate datasets	Treating scores as verdicts

> **Interview summary.** "I start with stack counting and first-seen because they're cheap and explainable. For network C2 I score interval CV, payload consistency and destination rarity. I use lineage and auth graphs to scope what I find, peer groups and time series to judge whether behaviour is unusual for *that* entity, and sequences to turn a confirmed pattern into a high-fidelity detection. Models and UEBA give me candidates; I give the verdicts."